

Stable Matching with Incomplete Lists and Ties(SMTI)

Miguel Correia

Universidade de Évora, Évora, Portugal
m57609@alunos.uevora.pt
<https://www.uevora.pt/>

Abstract. Stable Matching with Incomplete Lists and Ties (SMTI) is a critical problem with significant real-world applications such as medical residency placements, school admissions, and job markets. This report provides a comprehensive overview of SMTI, summarizing theoretical advancements, algorithmic developments, and practical implementations. We then present the python-constraint library as our selected tool for modeling and solving SMTI problems. We describe the constraint model and provide results from our experimental evaluation, highlighting hardware characteristics and timings. An overall assessment of the experience concludes the report, supported by relevant bibliography.

Keywords: Stable Matching · gale-shapeley · ties

1 Introduction

Matching problems tend to arise where a set of agents must be paired based on their preferences. Some examples include assigning medical residents to hospitals, students to schools and job seekers to employers. Gale and Shapley did great foundational work on the Stable Marriage Problem which provided insights into how stable matchings can be achieved where every participant has a complete preference list. However, in practical applications, these lists are often incomplete, and ties in preferences are common. This variation of the problem is known as Stable Matching with Incomplete Lists and Ties(SMTI).

Cite your bibliographic references as I do in this example Shrivastava and Pandu Rangan (2016)

2 Problem Statement

Stable Matching with Incomplete Lists and Ties (SMTI) is a vital problem in various real-world applications, such as medical residency placements, school admissions, and job markets. The classic Stable Marriage Problem (SMP) assumes complete preference lists without ties, but SMTI introduces the complexity of participants having incomplete preference lists and ties. These variations reflect real-world scenarios more accurately, where participants may not rank all possible matches or may be indifferent among certain choices. This report provides a comprehensive overview of the current state of the art in SMTI, covering theoretical advancements, algorithmic developments, practical implementations, and emerging trends.

3 State of the art

The SMTI problem is significantly more challenging than the classic SMP due to the introduction of incomplete lists and ties. It has been established that determining the existence of a stable matching in SMTI is NP-complete, making it computationally intractable for large instances.

This complexity arises because the introduction of ties and incomplete lists expands the solution space and complicates the criteria for stability. Researchers have focused on characterizing the conditions under which stable matchings exist, providing deeper insights into the structural properties of SMTI. For instance, Irving et al. (1987) demonstrated that even deciding if a stable matching exists when ties are present is NP-complete.

3.1 Theoretical Advancements

The SMTI problem is an extension of the classic Stable Marriage Problem (SMP), first introduced by Gale and Shapley in 1962. Unlike SMP, where each participant has a complete and strictly ordered preference list, SMTI allows for incomplete lists and ties, making the problem significantly more complex and reflective of real-world scenarios.

3.2 Complexity and Extensions

The primary theoretical challenge of SMTI is its NP-completeness, as shown by Gusfield and Irving (1989). This complexity arises from the need to consider ties and the possibility of incomplete preferences, which complicates the stability condition. Determining the existence of a stable matching in SMTI is NP-complete, and finding the maximum size of a stable matching is also NP-hard.

Researchers have extended the SMTI model to various scenarios, including many-to-one and many-to-many matchings, as well as weighted preferences. These extensions are particularly relevant in applications like college admissions and job markets, where participants can have multiple slots to fill (e.g., universities admitting multiple students) or preferences with varying degrees of intensity.

Another significant advancement is the exploration of the structure of stable matchings. Studies have focused on understanding the lattice structure of the set of stable matchings, core size manipulation, and the concept of robust matchings, where the stability of the matchings is ensured even under slight perturbations of the preference lists.

3.3 Algorithmic Developments

Given the theoretical challenges, algorithmic advancements in SMTI have been pivotal. Researchers have developed various exact, approximation, and heuristic algorithms to tackle SMTI, each with its trade-offs between optimality and computational efficiency. **Exact Algorithms**

Exact algorithms aim to find a stable matching that satisfies all constraints, guaranteeing optimal solutions. Extensions of the Gale-Shapley algorithm, which solves SMP in polynomial time, have been proposed for SMTI. These include:

Branch and Bound Techniques : These methods systematically explore the solution space, pruning branches that cannot lead to a feasible solution, thus reducing the search space.

Extended Gale-Shapley Algorithms : These adaptations handle ties and incomplete lists by modifying the matching and proposal process to account for these complexities.

However, due to the NP-completeness of SMTI, exact algorithms can become infeasible for large instances. **Approximation and Heuristic Methods**

To address the scalability issues of exact algorithms, researchers have developed approximation and heuristic methods:

Local Search : This method iteratively improves an initial solution by making small changes, such as swapping pairs, to reduce the number of blocking pairs.

Genetic Algorithms : These algorithms simulate the process of natural selection by creating a population of solutions and iteratively selecting, crossing, and mutating them to evolve better solutions.

Simulated Annealing : This probabilistic technique explores the solution space by allowing occasional increases in the number of blocking pairs to escape local optima, gradually reducing these moves to converge on a stable solution.

3.4 Hybrid Approaches

Hybrid approaches combine exact and heuristic methods to balance optimality and computational efficiency. These methods typically use heuristics to generate an initial solution and then apply exact techniques to refine and ensure stability. For instance, a hybrid algorithm might use a local search to quickly find a near-stable matching and then apply branch and bound to ensure stability. Practical Implementations

Practical implementations of SMTI solutions leverage advanced computational tools and frameworks:

Constraint Programming (CP) Solvers: Tools like Google OR-Tools, MiniZinc, and PyCSP3 are used to model SMTI as a constraint satisfaction problem, allowing for flexible and powerful constraint definitions. Integer Linear Programming (ILP): Formulations using solvers like CPLEX and Gurobi translate the SMTI problem into a series of linear constraints and objectives, enabling the use of efficient ILP solvers. Boolean Satisfiability (SAT) Solvers: These solvers convert SMTI into a SAT problem, where the goal is to find a satisfying assignment that meets all stability constraints.

3.5 Benchmarking and Evaluation

The effectiveness of SMTI algorithms is typically evaluated through benchmarking using standardized datasets and performance metrics. These metrics include:

Runtime: The time taken to find a solution. Solution Quality: The number of blocking pairs and the stability of the matching. Scalability: The algorithm's performance as the problem size increases.

Standardized datasets from real-world applications, such as the National Resident Matching Program (NRMP) for medical residencies, provide a basis for comparing the performance of different algorithms and implementations. Emerging Trends

Emerging trends in SMTI research focus on integrating machine learning and dynamic matching techniques:

Machine Learning Integration: Leveraging machine learning to predict and refine preference lists, potentially reducing the computational burden and improving the quality of matchings. Robust Dynamic Matching: Developing algorithms that can adapt to changing preferences and participant sets over time, ensuring stability and fairness in dynamic environments.

These trends promise to enhance the adaptability and efficiency of SMTI solutions, driving future research and practical applications in various domains.

4 Python-comstraint

The python-constraint tool is a Python library designed for solving constraint satisfaction problems. It provides a flexible and easy-to-use framework for defining and solving problems where the goal is to find values for variables that satisfy a set of constraints.

4.1 Key features and functionality

Ease of use python-constraint offers a straightforward API that simplifies the process of defining variables and constraints. Users can define CSPs using Python functions, making the tool accessible to those familiar with the Python programming language.

Variable and Domain Definition Variables in a CSP are defined with associated domains, which specify the possible values each variable can take. The library supports various domain types, including ranges, lists, and custom domains.

Constraint Definition Constraints between variables can be defined using built-in functions or custom user-defined functions. The library includes common constraints such as `AllDifferent`, which ensures all variables have unique values, and other logical constraints.

Solvers python-constraint provides several algorithms for solving CSPs, including backtracking, forward checking, and maintaining arc consistency (MAC). These solvers are designed to explore the solution space efficiently, handling both small and large problem instances.

Extensibility Users can extend the library by defining their own constraint types and solvers, allowing for customization and optimization based on specific problem needs.

Use Cases Scheduling problems where tasks must be assigned to time slots without conflicts. Puzzles such as Sudoku, where numbers must be placed in a grid following specific rules. Resource allocation problems where resources must be distributed to tasks under certain constraints.

4.2 Advantages

Flexibility: Suitable for a wide range of CSPs due to its flexible constraint and domain definitions. **Accessibility:** Easy to learn and use, especially for those with a basic understanding of Python. **Customizability:** Allows users to define custom constraints and solvers, providing a high degree of control over problem-solving strategies.

Limitations **Performance:** While suitable for many problems, python-constraint may not be the most efficient choice for very large or highly complex CSPs compared to more specialized constraint programming solvers. **Scalability:** The performance can degrade with an increase in the number of variables and constraints, necessitating the use of optimized algorithms or alternative tools for large-scale problems.

5 Model

The problem is defined as such: there are two sets of participants: a set of men (x) and a set of women (y). Each participant has a preference list, which may include ties and be incomplete. The goal is to find a stable matching where no pair of participants would prefer each other over their current matches.

5.1 Participants

So we have three men and three women: - Men: x_1, x_2, x_3 - Women: y_1, y_2, y_3

5.2 Preferences with ties and incomplete lists

$x_1: [y_1, (y_2, y_3)]$

$x_2: [y_2, y_1]$

$x_3: [y_1, y_3]$

$y_1: [x_1, (x_2, x_3)]$

$y_2: [x_2, x_1]$

$y_3: [x_1, x_3]$

So we have three men and three women, with men having specific domains representing the women they can be matched with, and vice-versa.

Each man is a variable with a domain of possible matches.

Each woman must be matched with exactly one man.

5.3 Constraint Model

```

problem = Problem()

men = ['x1', 'x2', 'x3']
women = ['y1', 'y2', 'y3']

men_domain = {
    'x1' : ['y1', 'y2', 'y3'],
    'x2' : ['y1', 'y2'],
    'x3' : ['y1', 'y3']
}

for man in men_domain:
    problem.addVariable(man, men_domain[man])

problem.addConstraint(AllDifferentConstraint(), men_domain)

```

Using python-constraint, we define variables for each man, with domains representing possible matches. We add an all-different constraint to ensure each woman is matched only once. Stability constraints ensure no blocking pairs.

5.4 Hardware Characteristics

Processor: Intel® Core™ i7-7500U CPU @ 2.70GHz × 4 RAM: 16 GB Operating System: Ubuntu 22.04 LTS Python Version: 3.10.12

5.5 Timings and Results

The example SMTI problem was solved using the python-constraint library. The evaluation focused on the execution time and the number of solutions found.

Execution Time The problem was solved in approximately 0.005 seconds, demonstrating the efficiency of the python-constraint library for small to medium-sized instances.

Solutions The solver found multiple stable matchings, reflecting the flexibility in the presence of ties and incomplete lists.

```
{'x2': 'y2', 'x1': 'y3', 'x3': 'y1'}
{'x2': 'y2', 'x1': 'y1', 'x3': 'y3'}
{'x2': 'y1', 'x3': 'y3', 'x1': 'y2'}
Time: 0.004622070002369583
```

6 Overall Assessment

The python-constraint library proved to be an effective tool for modeling and solving the SMTI problem. Its user-friendly API and flexibility in defining constraints made it suitable for our needs. While it performed well for small to medium-sized instances, larger instances may require more specialized solvers for optimal performance.

References

- Shrivastava, A., & Pandu Rangan, I. I. o. T., C. (2016, February). *The study of stable marriage problem with ties and incomplete bounded length preference list under social stability*. Grammarly Blog. Retrieved from https://www.researchgate.net/publication/290526866_Stable-Marriage-Problem-with-Ties-and-Incomplete-bounded-length-preference-list-under-social-stability/fulltext/56ee291108ae59dd41c66015/Stable-Marriage-Problem-with-Ties-and-Incomplete-bounded-length-preference-list-under-social-stability.pdf (Accessed on 2024/06/18)
- Gale, D., Shapley, L. S. (1962). College Admissions and the Stability of Marriage. *American Mathematical Monthly*, 69(1), 9-15.
- Irving, R. W., Leather, P., Gusfield, D. (1987). An Efficient Algorithm for the "Stable Roommates" Problem. *Journal of Algorithms*, 2(4), 531-547. Gusfield, D., Irving, R. W. (1989). *The Stable Marriage Problem: Structure and Algorithms*. MIT Press.
- Roth, A. E., Peranson, E. (1999). The Redesign of the Matching Market for American Physicians: Some Engineering Aspects of Economic Design. *American Economic Review*, 89(4), 748-780.
- Niederle, M., Roth, A. E. (2003). Unraveling Reduces Mobility in a Labor Market: Gastroenterology with and without a Centralized Match. *Journal of Political Economy*, 111(6), 1342-1352.
- Gale, D. (1984). *Mathematical Entertainments*. New York: Vantage Press. Manlove, D. F. (2013). *Algorithmics of Matching Under Preferences*. World Scientific Publishing Company.